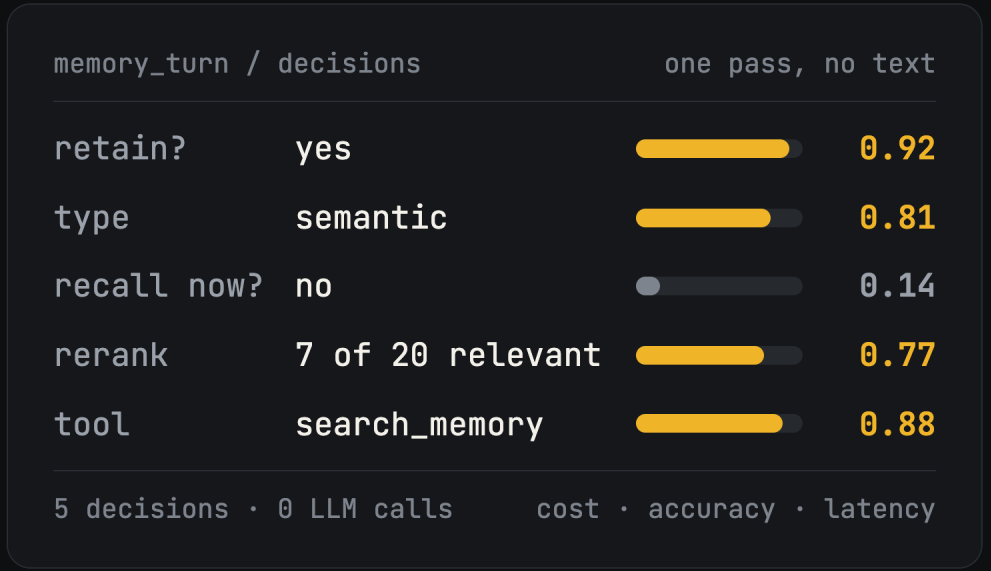
The Decision on Decision Models

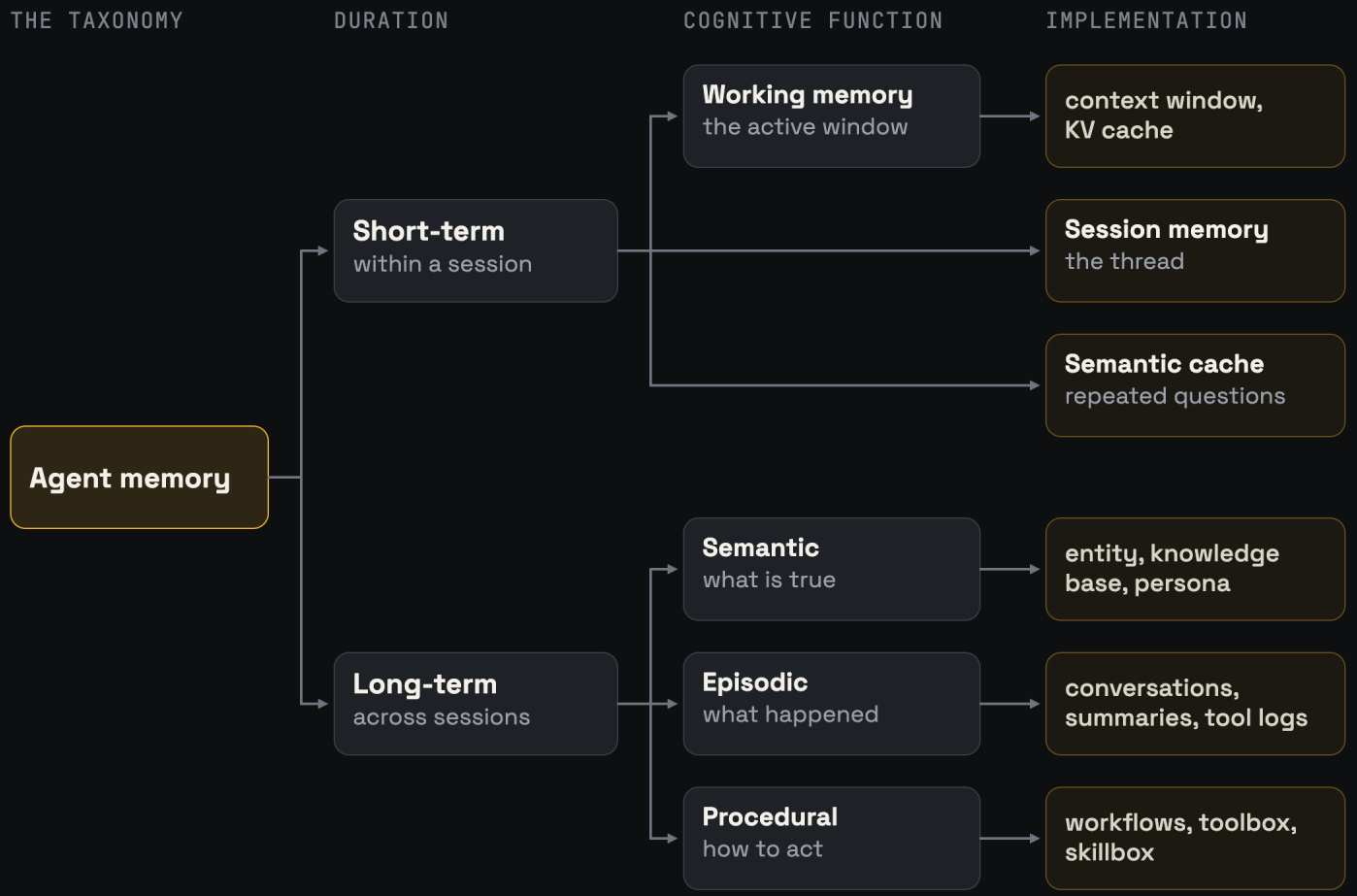
What They Do to Your Agent's Memory



with Richmond Alake

What agent memory is

Agent memory is how an agent **retains**, **reuses**, **refines** and **recalls** what it learns, across turns, sessions and agents, so it can see **long-horizon tasks** through.



WHERE IT LIVES

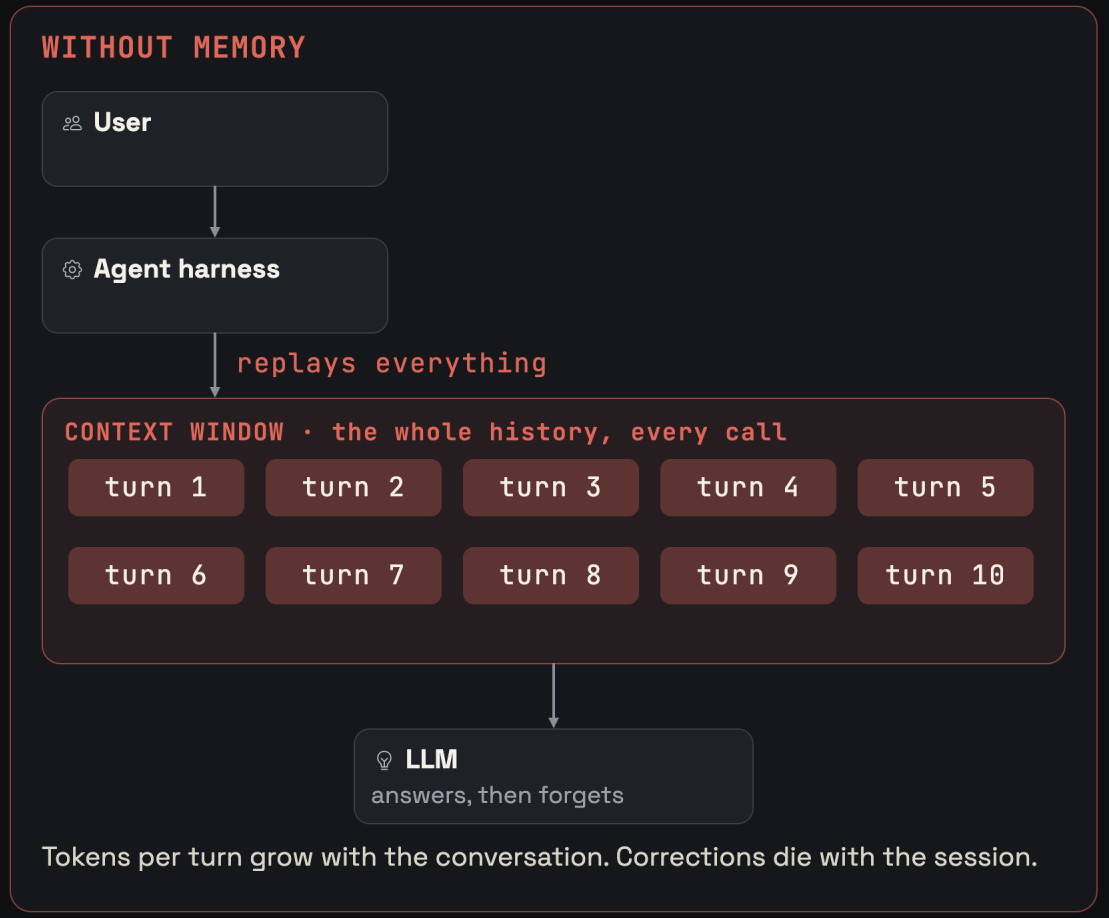
PARAMETRIC
Model weights
LLM or decision model; changed by fine-tuning

WORKING
Context window
this call, plus the KV cache

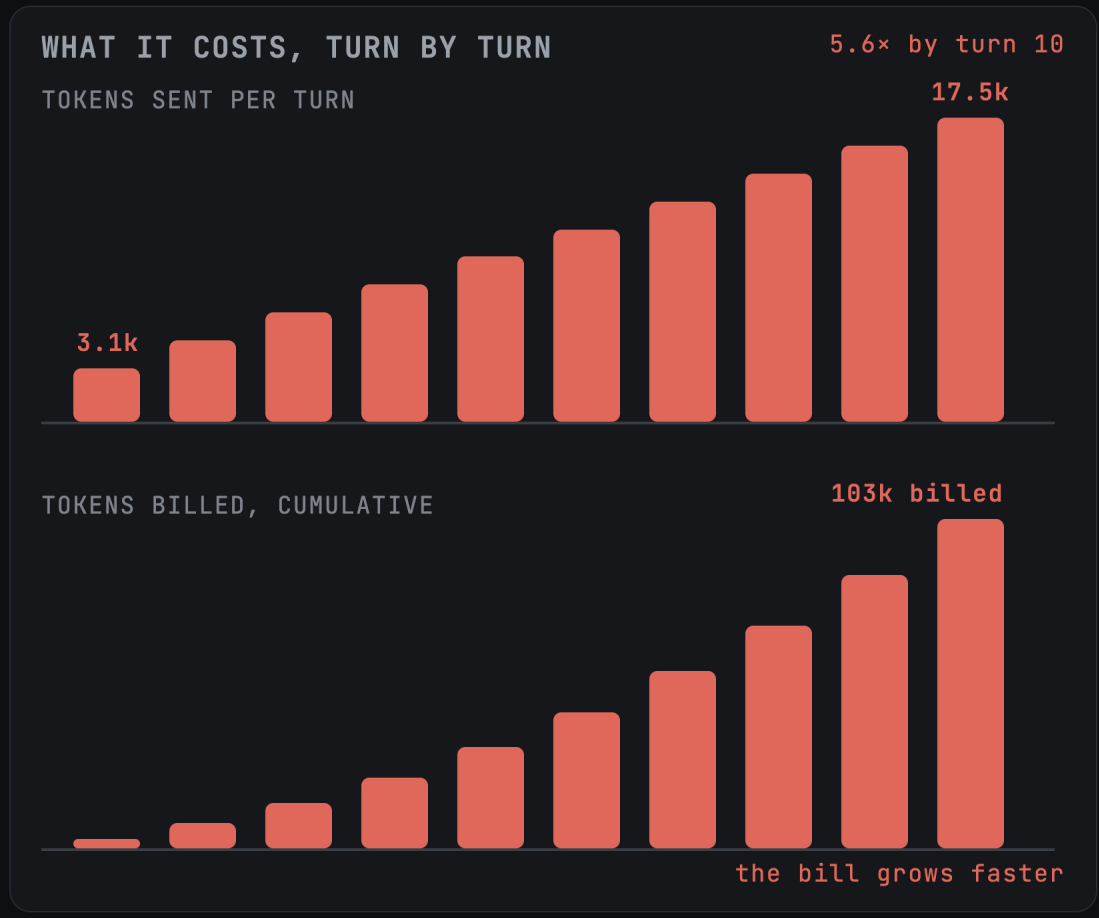
PERSISTENT
Database / Filesystem (external storage)
survives sessions and agents

01 · AGENT MEMORY · WITHOUT MEMORY

Without memory: the same loop, turn by turn

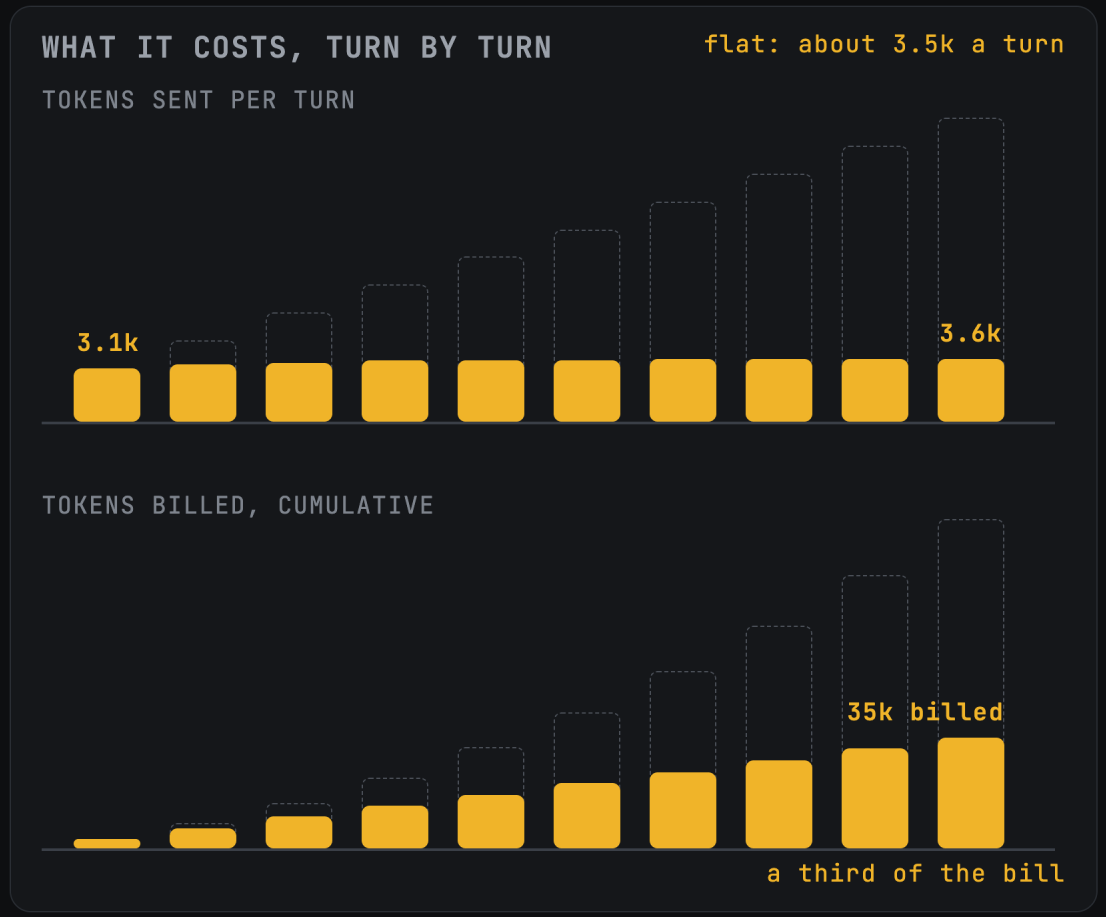


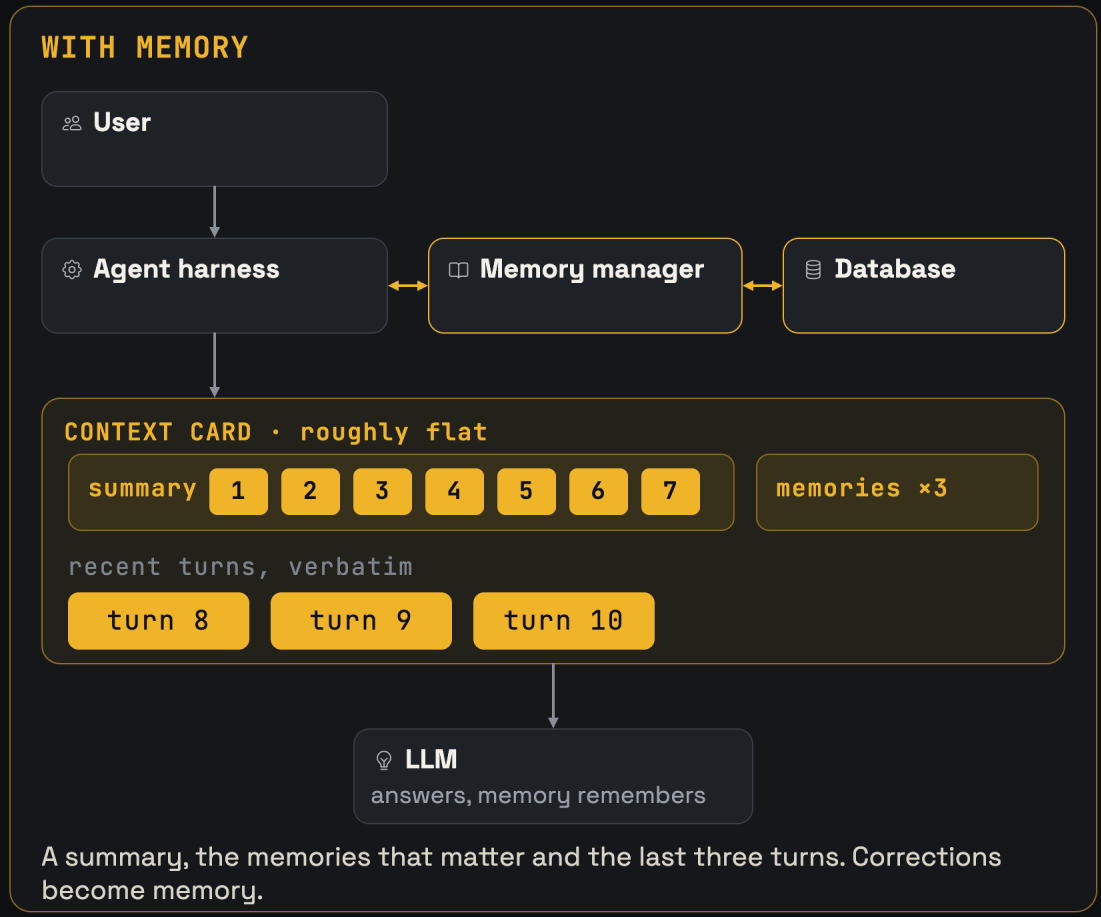
Tokens per turn grow with the conversation. Corrections die with the session.



01 · AGENT MEMORY · WITH MEMORY

With memory: the same loop, turn by turn

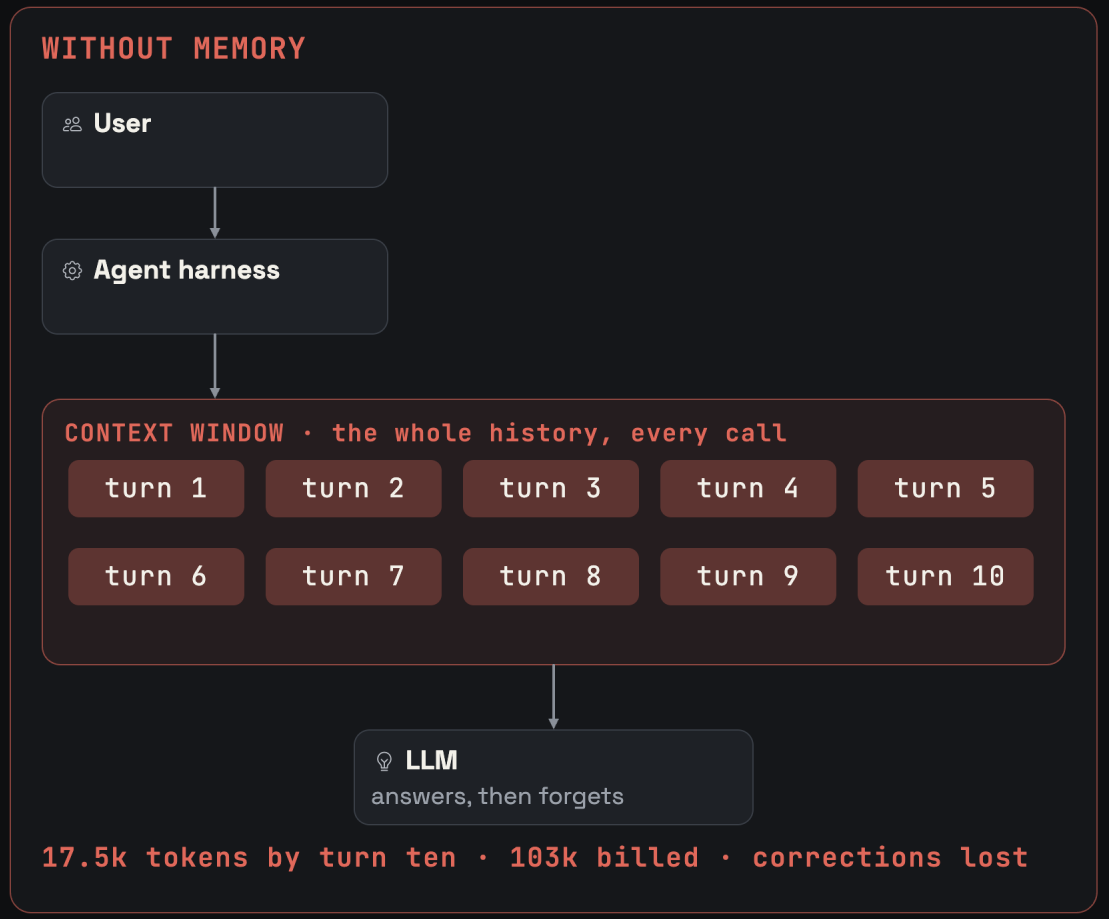


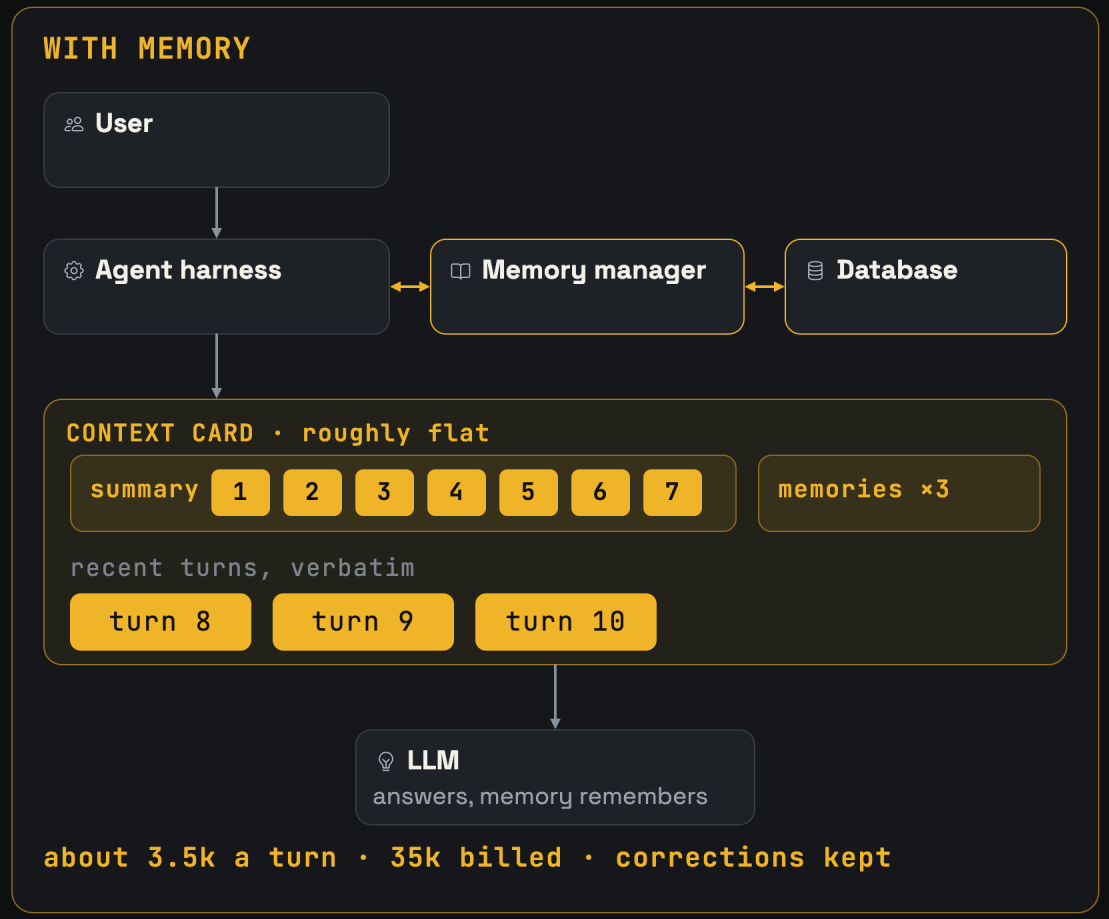


A summary, the memories that matter and the last three turns. Corrections become memory.

01 · AGENT MEMORY · SIDE BY SIDE

Side by side: the loop without and with memory





02 · PRINCIPLES

Three principles of agent memory

PRINCIPLE 1

The RBCs

Memory exists to make an agent more **reliable**, more **believable** and more **capable**. Every memory decision is judged against those three, not against how much the system stores.

reliability · believability · capability

PRINCIPLE 2

Forget, don't delete

A record that is wrong, stale or superseded is marked and demoted, never removed. Lossy summarisation with no archive behind it is deletion by another name.

mark it, demote it, keep it recoverable

PRINCIPLE 3 · THIS SESSION

The four Rs

Retain, refine, recall, reuse. The loop every memory system runs, whether or not anyone drew it. Each R is a stage of the pipeline, and each stage is full of decisions.

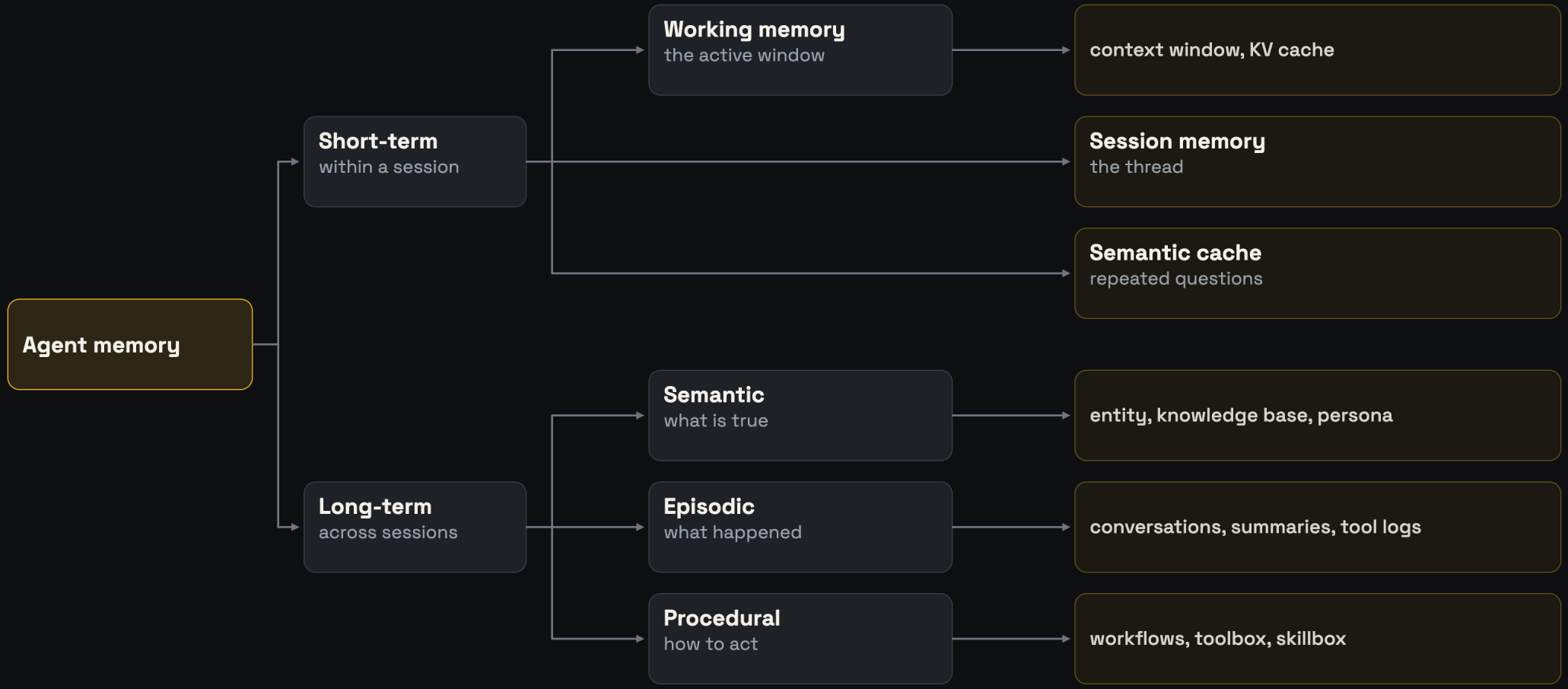
the frame for the rest of the session

prompt engineering is maximisation · context engineering is optimisation · memory engineering is continuation

03 · REFERENCE ARCHITECTURE

From the taxonomy to the architecture

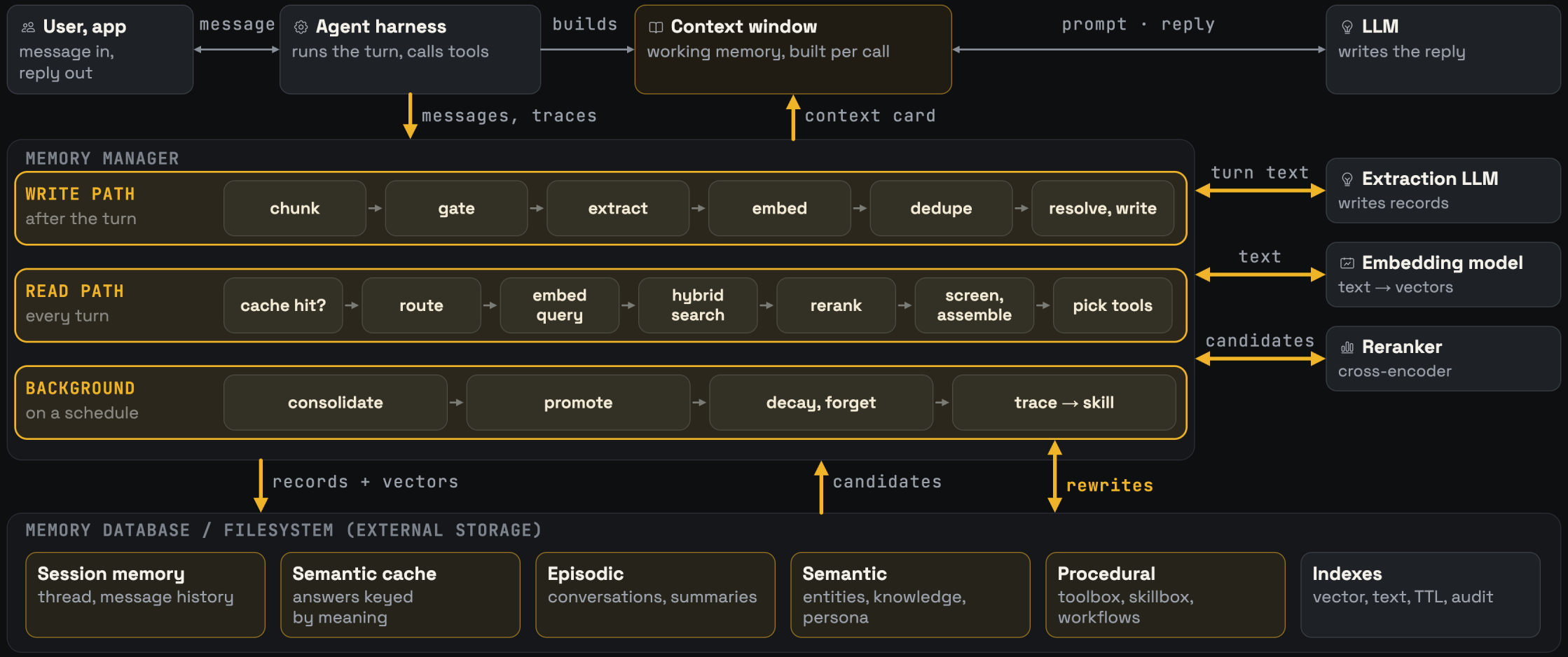
every type gets a place to live



03 · REFERENCE ARCHITECTURE

Agent memory reference architecture

data flows labelled



03 · TECHNIQUE · SEMANTIC CHUNKING

Semantic chunking: splitting where the meaning changes

Before anything is embedded, documents are cut into chunks. Semantic chunking embeds each sentence and splits wherever neighbouring sentences stop looking alike.

WRITE TIME · once per document, at ingestion



WHY IT MATTERS

Chunks are the unit of recall. Cut through an idea and retrieval returns half of it; merge two ideas and every hit drags in text the query never asked for.

THE DECISION INSIDE

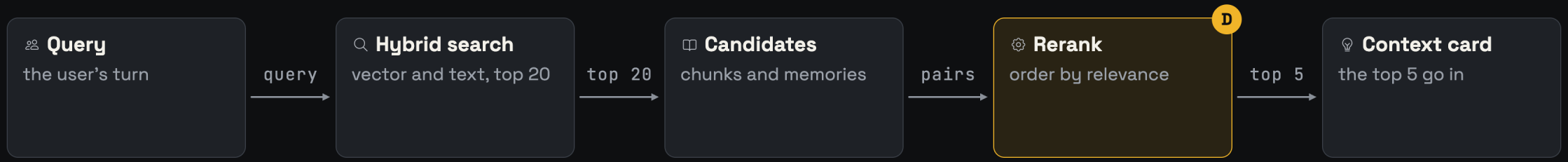
Does a new unit start here? Today a percentile cut on the similarity drop, set once per corpus. It is a yes or no for every gap between sentences.

03 · TECHNIQUE · RERANKING

Reranking: putting the best candidates first

Hybrid search is built for recall, so it returns more than the window can hold. A reranker scores each candidate against the query and keeps the top few.

EVERY TURN · on the hot path, after search and before the LLM



WHY IT MATTERS

The window is small and the model leans on what comes first. Whatever lands in the top few slots shapes the answer, so at this point ordering matters more than recall.

THE DECISION INSIDE

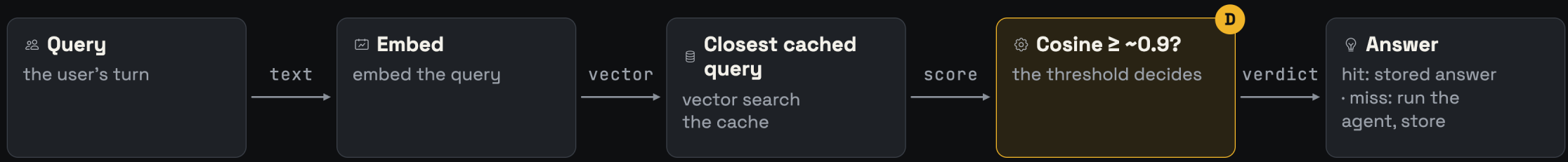
How likely is it that this candidate answers this query? Today cosine order, or a cross-encoder trained on someone else's data. A score per pair, then a sort.

03 · TECHNIQUE · SEMANTIC CACHE

The semantic cache, and where its false hits come from

A typical semantic cache embeds the incoming query, finds the closest cached query, and returns the stored answer if the cosine similarity is above something like 0.9.

EVERY TURN · on the hot path, before retrieval and the LLM



FALSE HITS: HIGH SIMILARITY, WRONG ANSWER

"Weather in London today" vs "… tomorrow"
"How do I cancel my order" vs "… my subscription"
"Revenue for Q2" vs "revenue for Q3"
Negation, different entities, different numbers

THE TRAP

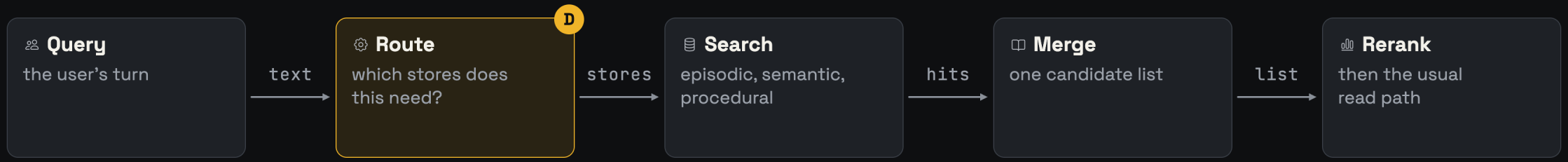
Each pair scores very high, but the cached answer is wrong for the new query. A wrong answer is worse than a miss, so teams set the threshold high and end up with a poor hit rate.

03 · TECHNIQUE · MEMORY ROUTING

Memory routing: sending each query to the right store

Long-term memory is several stores, not one. Routing decides which of them a query searches, and on the write side, which store a new memory lands in.

EVERY TURN · on the hot path, before search



WHY IT MATTERS

Search every store on every turn and you pay in latency and noise. Search the wrong one and the memory is there but never found.

THE DECISION INSIDE

Which memory types does this query need: episodic, semantic, procedural, or none? Today keyword rules or the nearest store description. A pick from a list, sometimes more than one.

03 · TECHNIQUE · CONSOLIDATION AND DEDUPLICATION

Consolidation and deduplication: keeping one version of each fact

Each new record is checked against what the store already holds, and on a schedule the store looks at itself: near-identical records are merged, and episodic notes that keep agreeing become a semantic fact.



IN PARALLEL · every turn carries on as normal: recall, reply, user; this only changes what is there next time

WHY IT MATTERS

Retain only ever adds. Without this, the store fills with paraphrases of one fact and recall returns five copies of it. With it, ten notes that the user prefers Python become one fact, with the ten kept as provenance.

THE DECISION INSIDE

Is this the same record, an update, something to merge, or new? Today a fixed cosine cut-off (memorizz uses 0.95) or an LLM prompt per pair or cluster. A pick from a short list.

03 · TECHNIQUE · MEMORY EXTRACTION

Memory extraction: turning a turn into typed records

After the reply is sent, the turn is a candidate memory. The LLM turns what is worth keeping into typed records: a preference, a fact, a procedure, an event.



WHY IT MATTERS

Store every turn and recall wades through “ok thanks”. Run a large extraction prompt on every turn and you pay for a long generation even when nothing is worth keeping.

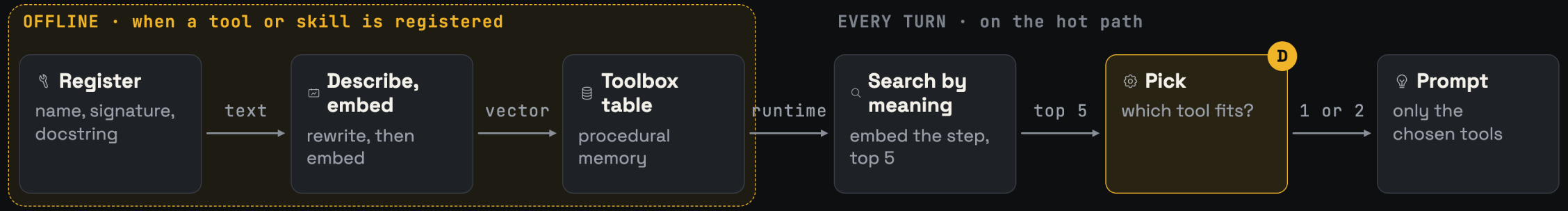
THE DECISION INSIDE

Is anything here durable, and if so what type? Today there is no decision: it always runs. A yes or no, then a pick from four types.

03 · TECHNIQUE · TOOLBOX AND SKILLBOX SELECTION

Toolbox and skillbox selection: tools retrieved by meaning

Tools and skills are procedural memory. Store them with good descriptions and embeddings, then put only the few that fit this step into the prompt.



WHY IT MATTERS

With every tool in every prompt, tokens grow with the registry and selection tends to get worse as the list grows. Skills (SKILL.md files) work the same way.

THE DECISION INSIDE

Which tool fits this step? Today an embedding top-k, then the LLM picks from what is left. It is a pick from a known list.

03 · REFERENCE ARCHITECTURE · THE DECISIONS

Every D is a decision

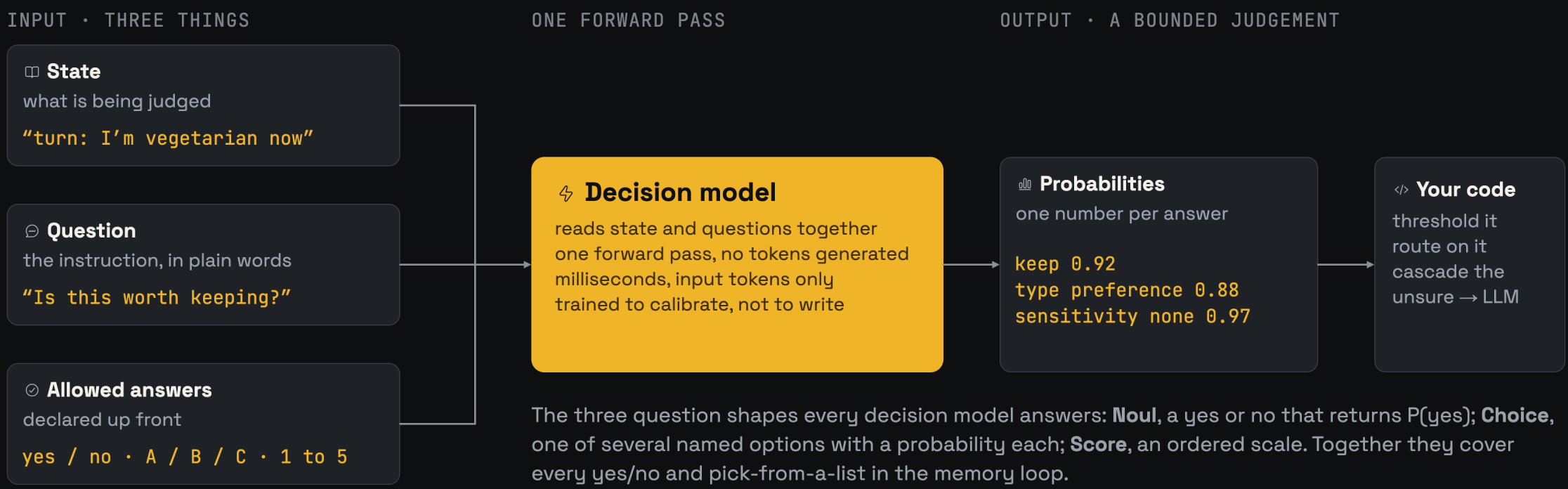
13 marked · 20 decisions



04 · DECISION MODELS

What a decision model is

A decision model is a neural network that takes a **state** (what is being judged), a **question**, and the **answers it is allowed to give**, and returns a probability for each answer in one forward pass. It writes no text, so your software can act on the result directly: threshold it, route on it, or pass the uncertain cases to an LLM.



The three question shapes every decision model answers: **Noul**, a yes or no that returns P(yes); **Choice**, one of several named options with a probability each; **Score**, an ordered scale. Together they cover every yes/no and pick-from-a-list in the memory loop.

04 · DECISION MODELS

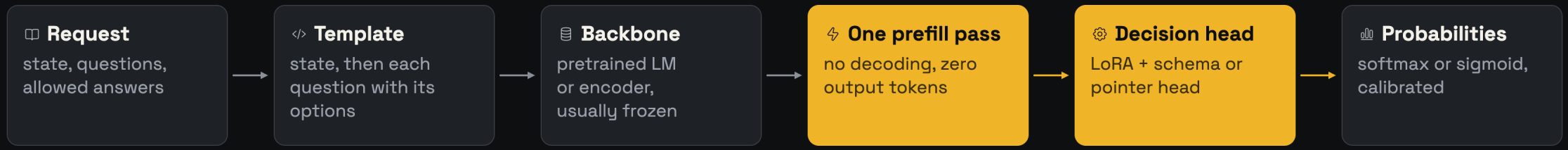
Classifier, LLM, decision model: what you give, what you get

	ML classifier	LLM	Decision model
What you give it	Text or features; the label set was fixed at training	A prompt: instructions, context and the question	State, question and allowed answers, in the request
What comes back	One label from that set, plus a softmax score	Free text you have to parse	A probability per allowed answer, no text
To ask a new question	Collect labels, retrain, redeploy	Write a new prompt	Change the question in the request; no training
Confidence	Softmax score, often overconfident off-distribution	Whatever the text says it is	A calibrated probability you can threshold
Latency and cost	Milliseconds, near zero	Seconds; tokens in and tokens out	Milliseconds; input tokens only
Where it fits in the loop	One job each: a spam filter, a PII tagger	Reply, extraction, merge: the generation steps	Every yes/no and pick-from-a-list: the 20 points

A decision model sits between the other two: the speed and bounded output of a classifier, with the question supplied at request time like a prompt. It keeps the LLM for the work only a generator can do.

04 · DECISION MODELS · ARCHITECTURE

How a decision model is built



Why it is fast: the state is read once, every question is answered from that one pass, and nothing is generated. Why it is useful: each answer is a probability over options you declared, so it can be thresholded.

FIVE WAYS THE FIELD BUILDS THE HEAD

A · LM-head scoring

Read the logits of each option's tokens from an ordinary LLM. No training; weakest calibration.

`any instruct LLM`

B · Frozen backbone, trained head

Keep the LLM frozen, train adapters and a head over the hidden states with a calibration loss.

`Clef: Qwen3.8-27B, 256 MB head, Brier loss`

C · Pointer head

A small head points at the span of the chosen option in the input.

`Strands Decider 2B: Qwen3.5-2B, small LoRA`

D · Encoder classifier

A bidirectional encoder reads state and options together. Smallest and fastest.

`Laya 421M (ModernBERT), d1-omni-600M`

E · Dual encoder

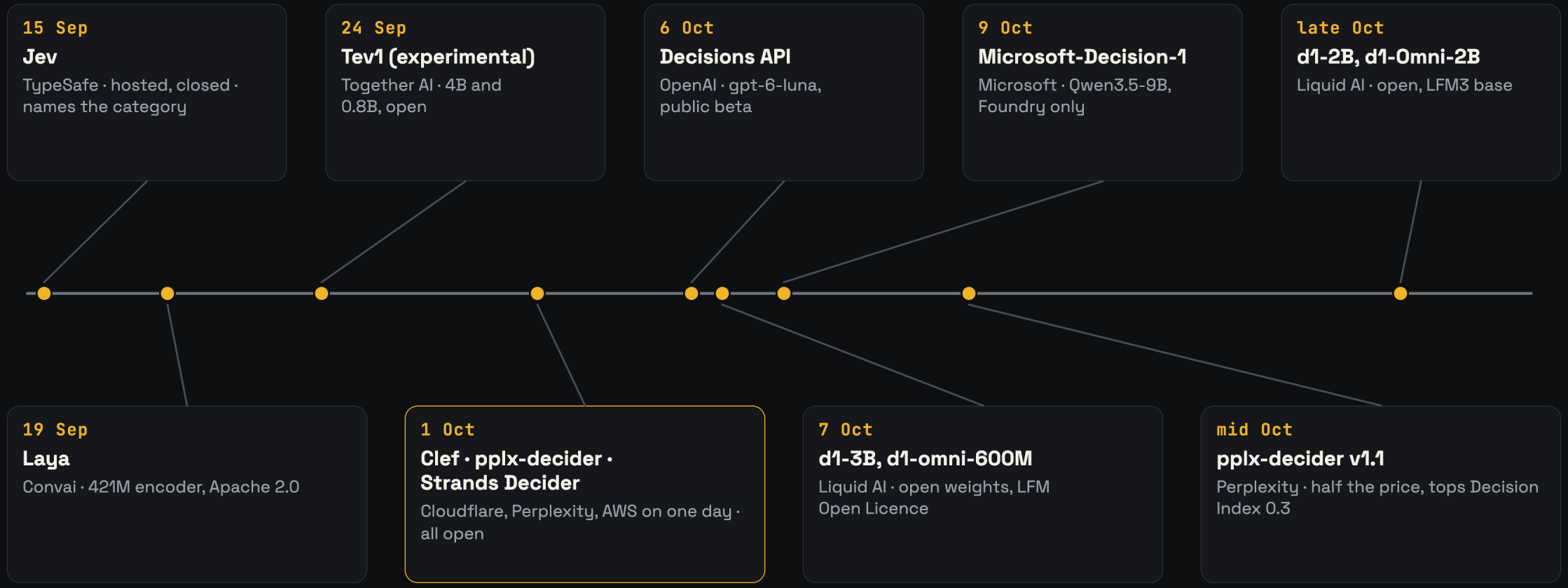
Encode the state and each action separately; cache the action side, score by similarity.

`CLM-8B: InfoNCE, cached actions`

Training data is labelled decisions; the loss rewards calibrated probabilities, not just the right answer. Jev adds reinforcement learning on calibrated decisions (RLCD).

04 · DECISION MODELS · THE SYSTEM 1 ECOSYSTEM

Seven weeks, ten launches, one request shape



Nearly all answer the same /v1/systemone request shape (Noul, Choice, Score), so the model under the architecture is a configuration choice. Output is free everywhere; you pay for input.

05 · OPERATIONS · RETAIN

Retain: the write path, after the turn

8 decisions · LLM 6 · rules 2

STAGE 1 · SPLIT

Split the turn

1 · Chunk boundary
Does a new unit start here?
today: cosine drop
later: Noul

then: each unit becomes a candidate memory

STAGE 2 · GATE

Decide what to keep

2 · Retention gate
Worth keeping at all?
today: LLM
later: Noul, p = salience

3 · Memory type
Which memory type?
today: LLM
later: Choice

4 · Sensitivity gate
Must this never be stored?
today: regex, then LLM
later: Choice, masked

then: LLM extracts the record

STAGE 3 · CHECK

Check what is stored

5 · Extraction quality
Is the record faithful?
today: LLM as judge
later: Noul

then: vector search, top-k

6 · Duplicate check
Same as an existing record?
today: cosine threshold
later: Noul per candidate

7 · Conflict detection
Contradicts an existing one?
today: LLM
later: Noul per candidate

STAGE 4 · WRITE

Write the record

8 · Resolution
Insert, update, merge or forget?
today: LLM
later: Choice

then: embed, write the record and its vector. Forget, don't delete: the old record is marked, not removed

Six LLM judgements per unit before anything is written, plus the extraction call. Later: 2 to 4 are one request, 6 to 8 are one request.

05 · OPERATIONS · REFINE

Refine: background jobs over the store

4 decisions · LLM 2 · rules 2
off the hot path

STAGE 1 · SCORE

Which records went stale?

9 · Staleness
Is this still true, or past its date?
`today: age decay, a rule`
`later: Noul`

11 · Memory utility
Did this record help the turn it was used in?
`today: access counts, a rule`
`later: Noul`

utility is a running tally of those per-turn verdicts; low scores are marked, not deleted

STAGE 2 · RESHAPE

Merge and promote

10 · Consolidation, promotion
Merge, promote, or leave alone?
`today: LLM`
`later: Choice`

then: the LLM writes the merged record, one generation call per merge. Deciding whether to merge and writing the merge are different jobs; only the first is a decision

STAGE 3 · LEARN

Turn traces into skills

12 · Trace selection
Should this trace become a skill?
`today: LLM`
`later: Noul`

then: selected traces are distilled into procedural memory and land in the tool registry, where Reuse can find them

Refine runs on a schedule, so nobody waits on it. Cost at scale is the lever: every record is rescored, every cluster gets a verdict.

05 · OPERATIONS · RECALL

Recall: the read path, on every turn

6 decisions · LLM 4 · rules 1 · reranker 1
all on the hot path

STAGE 1 · BEFORE RETRIEVAL

Whether and where to look

13 · Recall gate
Does this turn need memory at all?
`today: always retrieve`
`later: Noul`

14 · Memory routing
Which store or memory type?
`today: LLM`
`later: Choice, benchmarked`

15 · Query rewrite need
Is the raw query good enough?
`today: LLM`
`later: Noul`

then: rewrite if needed, embed, search

STAGE 2 · RANK

Keep what is relevant

16 · Reranking
Is this candidate relevant?
`today: cross-encoder, per pair`
`later: Noul each, or Score`

then: sort, cut at k. The reranker is the one model in the loop that already has exactly one job, so this is the fairest fight, and it is benchmarked

STAGE 3 · GATE THE RESULT

Trust what comes back

17 · Sufficiency check
Enough to answer, or search again?
`today: LLM`
`later: Noul`

18 · Injection screen
Does a memory carry instructions?
`today: LLM or classifier`
`later: Noul`

then: context assembly, budget, order, format

Four LLM judgements before the reply starts: routing, rewrite, sufficiency, screen. That is the latency the user feels. Later, 13 to 15 are one request.

05 · OPERATIONS · REUSE

Reuse: procedural memory into action

2 decisions · LLM 1 · rules 1

STAGE 1 · SELECT

Pick the tool or skill

19 · Tool and skill selection

Which tool or skill fits this step?

`today: LLM picks from the prompt`

`later: Choice, or CLM over cache`

then: or an embedding top-k pre-filter first, then the LLM picks. Either way the registry sits in the prompt, tokens grow with the number of tools, and the pick is text you parse

STAGE 2 · GUARD

Should the call run?

20 · Tool-call guard

Should this call run as written?

`today: allowlist rules`

`later: Noul, optional`

then: the harness executes the call and records the result

STAGE 3 · LOOP

Close the loop

No decision here, just plumbing

The trace of the call, with its outcome, goes back to Retain as a candidate procedural memory. Decision 12 decides later whether it becomes a skill.

`reuse feeds retain`

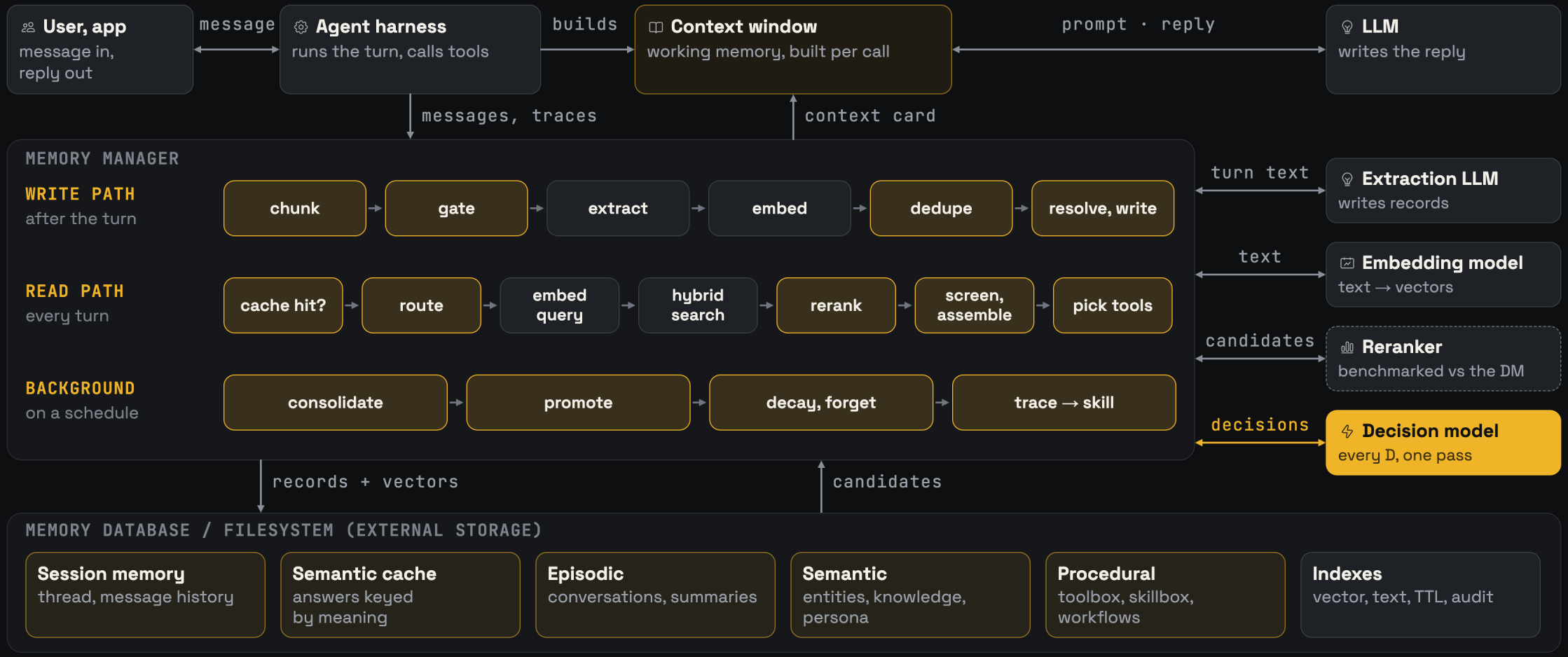
this is why the four Rs are a loop and not a list

Two decisions, but the first is the most expensive line in the prompt: every tool description, every turn, needed or not.

06 · WITH A DECISION MODEL

The same architecture, with a decision model

every D now goes to one model



Semantic chunking: a similarity drop vs “does a new unit start here?”

TODAY

Sentences
split the document

→

Embed
one vector per sentence

→

Neighbour cosine
sentence i vs sentence i + 1

→

Percentile cut
split where the similarity drops below it

WHERE IT BREAKS

“## Short-term memory … ## Long-term memory …”
`two sections, merged into one chunk`
They share most of their vocabulary (memory, agent, store, recall), so the similarity never drops and the boundary is missed.

“```python … ```” and the paragraph explaining it
`one unit, split apart`
Code and prose look very different as embeddings, so the drop is sharp and the explanation lands in a different chunk from its code.

WITH A DECISION MODEL

Sentences
the same split

→

Window
a few sentences either side of gap i

→

Decision model
binary: does a new self-contained unit start at sentence i?

→

Cut
where p ≥ your threshold

→

Embed chunks
once, the finished chunks

Runs once at ingestion, not at query time, so the extra cost is spread over every later read.

primitive: binary · runs at write time · measure: retrieval hit rate and chunk utilisation vs fixed and percentile chunking

06 · THE SWAPS · 2 OF 7 · RECALL

Reranking: on topic vs “does this chunk answer the query?”

TODAY

Query
“papers that criticise RAG”

→

Vector search
top 20 by cosine

→

Cross-encoder
optional: rescore each pair

→

Sort
keep the top k

WHERE IT BREAKS

“papers that criticise RAG”
`pro-RAG papers rank highly`
They are about RAG, so they score high. Similarity measures topic, not stance or intent.

“memory designs that avoid vector databases”
`vector database papers rank first`
Same failure: the query names what it wants left out, and similarity rewards the mention.

WITH A DECISION MODEL

Query
the same query

→

Vector search
the same top 20

→

Decision model
probability: how likely is it that this chunk answers this query?

→

Sort by p
keep the top k

Use the probability, not a yes or no: a binary answer throws away the ordering. A reranker trained on your domain is a strong baseline, and my own swap gave mixed results, so measure against it rather than assume the decision model wins.

primitive: probability (score) · runs every turn, on the hot path · measure: recall@k and nDCG vs your domain reranker, ms for 20 candidates

06 · THE SWAPS · 3 OF 7 · RECALL

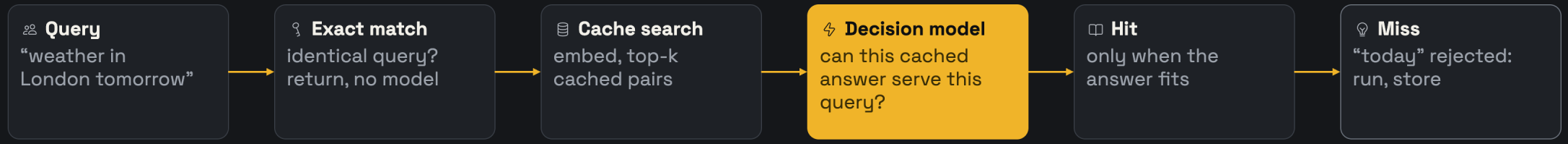
Semantic cache: a cosine threshold vs a decision model

TODAY · THE THRESHOLD DECIDES REUSE



The pair scores very high and the answer is wrong. To stop that, teams set the threshold high and the hit rate falls.

WITH A DECISION MODEL · EMBEDDINGS FIND CANDIDATES, THE MODEL DECIDES REUSE



Compare the new query with the cached answer, not the cached query: today vs tomorrow is rejected, and two different questions one answer covers still hit.

measure: false-hit rate on near-miss pairs (today/tomorrow, order/subscription, Q2/Q3) · hit rate · ms per lookup · cost per turn

The catch: every lookup now pays for a decision

The point of a cache is saving time and money. With a decision model in it, you pay for one call on every lookup, including the ones that miss.



COST PER LOOKUP

```
decision model + (miss rate × LLM)
compared with
LLM on its own

pays off when hit rate > decision model cost ÷
LLM cost
```

LATENCY

```
hit: one decision call, milliseconds
miss: the decision call, then the full LLM call

so a miss gets slightly slower; the exact-match tier
keeps identical queries free
```

A small model like d1 is far cheaper than a full generation, so this usually pays off once the hit rate is reasonable. If the hit rate is very low, it does not.

06 · THE SWAPS · 4 OF 7 · RECALL

Memory routing: nearest store vs “which memory types does this need?”

TODAY

Query
the user’s turn

→

Rules or embed
keywords, or embed the query

→

Nearest store
closest store description wins

→

One store
search that store only

WHERE IT BREAKS

“What did I tell you about the wedding last month?”
`sent to semantic memory`
The topic pulls it to the facts about the wedding, but “last month” and “I told you” mean it needs episodic memory.

“Book the same restaurant as last time; I’m vegetarian now”
`needs two stores, gets one`
Episodic for “last time”, semantic for the new preference. Nearest-store routing picks one and misses half the context.

WITH A DECISION MODEL

Query
the user’s turn

→

Decision model
choice, more than one label: which memory types does this query need?

→

Each chosen store
episodic, semantic, procedural, or none

→

Merge
candidates from every store

On the write side the same call decides which store a new memory goes into, so one question covers both directions.

`primitive: choice, multi-label · runs every turn and on write · measure: routing accuracy on labelled turns, including two-store turns`

06 · THE SWAPS · 5 OF 7 · RETAIN AND REFINE

Consolidation and deduplication: cosine > 0.95 vs a five-way choice

TODAY

New memory
from the write path

→

Vector search
most similar existing

→

Threshold
cosine > 0.95?

→

Merge or drop
the new memory

WHERE IT BREAKS

“prefers Python” vs “prefers Python for scripts, Go for services”
`merged, detail lost`
They score above the threshold, so the richer memory is merged away or dropped.

“lives in London” vs “moved out of London”
`a contradiction stored as a duplicate`
High similarity again: the new fact is dropped and the agent keeps saying London.

WITH A DECISION MODEL

New memory
from the write path

→

Vector search
top k similar

→

Decision model
choice: same · refines · contradicts · complements · unrelated

→

Action
skip · update · supersede · link · insert

same → skip · refines → update · contradicts → supersede · complements → link · unrelated → insert. Supersede marks the old memory instead of deleting it: forget, don’t delete.

primitive: choice · runs at write time and in background consolidation · measure: agreement on labelled pairs, contradiction recall

06 · THE SWAPS · 6 OF 7 · RETAIN

Memory extraction: always run vs a gate first

TODAY

Turn ends
"ok thanks" or "I'm vegetarian"

→

Always run
store every turn, or a large extraction prompt on every turn

→

Write
whatever comes back

WHERE IT BREAKS

"ok thanks" · "sure" · "got it"
`filler fills memory`
Storing every turn keeps them as memories that recall has to wade through later.

"Good morning, can you check my calendar?"
`the big prompt finds trivia`
Asked to find something, it does: "user says good morning" becomes a memory, and every turn pays for a large generation.

WITH A DECISION MODEL

Turn ends
after the reply

→

Decision model
binary: anything durable worth remembering?

→

Decision model
choice: preference · fact · procedure · event

→

LLM extraction
only on turns that pass

→

Write
typed records

This one is not replacing a similarity score. It replaces "always run it", which is another hidden heuristic. The expensive call only runs on turns that pass the gate.

`primitive: binary, then choice · runs after the turn, in the background · measure: turns extracted, precision of kept memories, cost per 100 turns`

06 · THE SWAPS · 7 OF 7 · REUSE

Tool and skill selection: top-k vs “which one, if any?”

TODAY

Request
the user’s turn

→

Embed
request and tool descriptions

→

Top k tools
by similarity to the request

→

Into context
the agent picks and calls

WHERE IT BREAKS

“Send the Q3 report to Sarah”
`generate_report ranks above send_email`
The request is about a report, so the report tool is closest; the action it needs is sending.

“Don’t email it, just post it in Slack”
`the email tool still comes back`
Negation barely moves the embedding, so send_email stays in the top k and gets called.

WITH A DECISION MODEL

Request
the user’s turn

→

Embedding top k
recall over a large tool catalogue

→

Decision model
choice: which of these tools does this request need, if any?

→

Into context
only the chosen tools, or none

Keep the embedding top k for recall. Including “none” as an option also cuts down on tools being called when they shouldn’t be.

primitive: choice with a none option · runs every turn · measure: top-1 tool accuracy, false tool calls, prompt tokens per turn

06 · THE SWAPS · THE PATTERN

Index for recall, predicate for precision

RECALL STAYS GEOMETRIC

ANN, BM25 and metadata filters are cheap, can be precomputed, and scale to millions of items. Decision models don't replace this step.

THE DECISION BECOMES A PREDICATE

A similarity score answers "how close are these?", a stand-in for the question you actually have. The decision model asks that question directly.

Use case	What similarity measures	What the predicate asks	Primitive
Chunking	Are neighbours close?	Does a new unit start here?	binary
Reranking	Is the chunk on topic?	Does it answer the query?	probability
Semantic cache	Is the query close?	Can this answer serve this query?	binary
Routing	Nearest store	Which stores does this need?	choice, multi-label
Consolidation, dedup	Are these near-identical?	Same, refines, contradicts…?	choice
Extraction	nothing: always run	Is anything here durable?	binary, then choice
Tool, skill selection	Is the tool on topic?	Which tool, or none?	choice

06 · THE SWAPS · WHERE IT PAYS

Two rules of thumb, and where it doesn't pay

RULE 1

Strongest where a false positive is costly

A wrong merge, a wrong cache hit, a wrong tool call. Those are the places where similarity says yes and the answer is no.

FOR EXAMPLE

- Dedup: "moved out of London" merged into "lives in London"
- Cache: today's weather served for tomorrow
- Tools: send_email called when the user said Slack

RULE 2

Decide at write time where you can

Chunking, extraction, dedup and routing on write are paid for once when data comes in, not on every read, and they create structure that reduces how much geometry you need at read time.

FOR EXAMPLE

- Write time, paid once: chunking, extraction, dedup, routing a new memory
- Read time, paid every turn: reranking, routing a query, tool choice, cache lookup

WHERE IT DOESN'T PAY

First-stage recall, and cheap mistakes

First-stage recall over large N, and hot paths with tight latency budgets where a wrong answer costs little.

FOR EXAMPLE

- Searching millions of memories or chunks: keep ANN and BM25
- A "related memories" hint where a miss costs little: cosine is fine
- Every turn under a tight budget: measure the ms first